

IT1244

Artificial Intelligence: Technology & Impact

Week 11 Tutorial 9

TODAY'S AGENDA

1. Quick Recap
2. Discussion of tutorial questions
3. Sharing by Aaron and Cheng Hou
4. Say goodbye :(

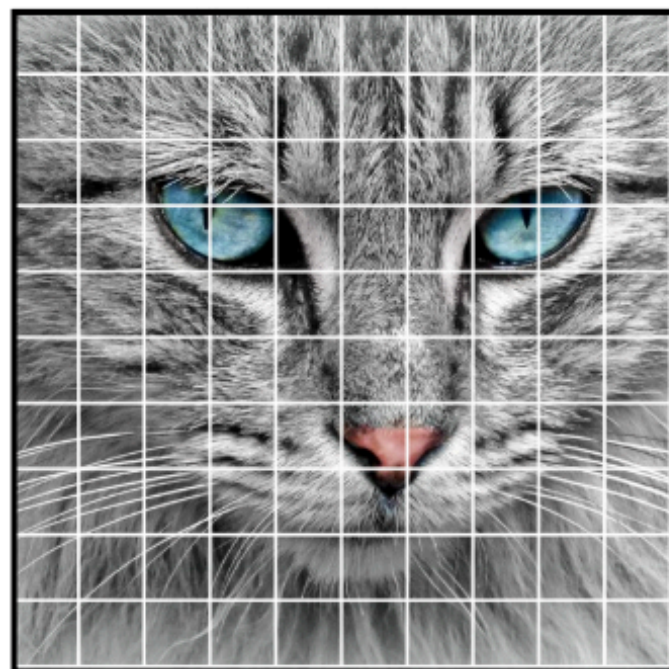


RECAP

CONVOLUTIONAL NEURAL NETWORK (CNN)

a type of deep learning model designed to process image data by learning spatial features

but first! we need to know that a coloured image is made up of pixels and channels (RGB)



192	206	187	132	163	174	176	167	162	230
162	161	110	194	113	209	56	156	182	149
131	152	174	51	2	148	145	210	167	191
57	115	159	13	30	122	105	172	59	183
107	120	49	86	65	116	162	44	184	115
87	126	119	114	116	142	217	112	102	88
95	70	115	193	165	80	137	161	129	108
126	108	82	213	154	206	26	55	162	120
187	149	111	35	115	84	107	106	121	167
160	121	127	68	107	56	91	113	157	164

R

194	208	187	133	164	175	178	169	164	232
164	161	110	194	113	209	56	156	182	149
131	152	173	51	2	148	145	210	167	191
57	115	158	26	30	122	105	166	112	183
107	120	49	85	65	116	162	44	184	115
87	126	119	114	116	142	217	112	102	88
95	71	115	193	165	81	139	161	129	108
126	108	82	213	154	206	21	55	162	120
187	149	111	35	115	84	107	106	121	167
162	123	129	71	109	58	92	115	159	166

G

193	207	187	133	164	175	177	168	163	231
163	161	110	194	113	209	56	156	182	148
131	152	173	51	2	148	145	219	167	191
57	115	155	24	29	122	105	166	130	183
107	120	49	83	65	116	162	44	184	115
87	126	119	114	116	142	217	112	102	88
95	69	115	193	165	81	138	161	129	108
126	108	82	213	154	206	19	55	162	120
186	149	111	35	115	84	107	106	121	167
161	122	128	69	108	57	92	114	157	165

B

credits to SDS workshop member Jason for this crazy visualization

WHY CNN? WHY NOT JUST USE A REGULAR NEURAL NETWORK?

A 224×224 colour image has $224 \times 224 \times 3 = 150,528$ input values. If your first hidden layer has 1,000 neurons, that's already 150,528,000 parameters (weights to train just for the first layer)

This is:

- Computationally expensive
- Takes up a lot of time
- And it still doesn't work well, because it treats every pixel independently with no sense of spatial structure



Spatial Structure:

Pixels that are close to each other are related

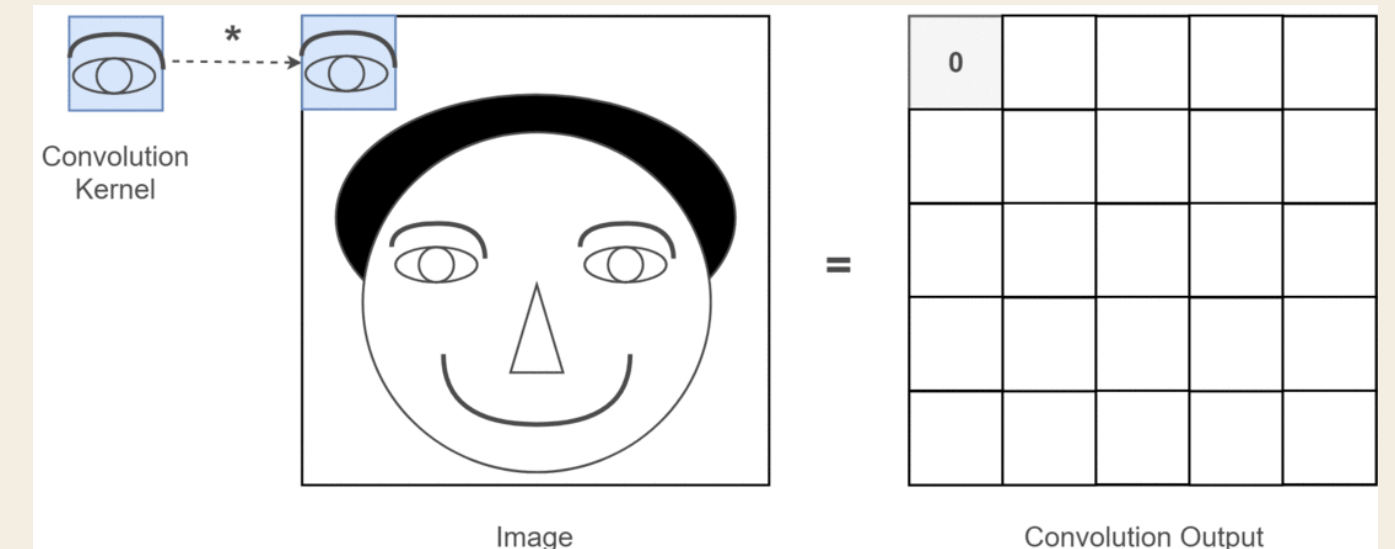
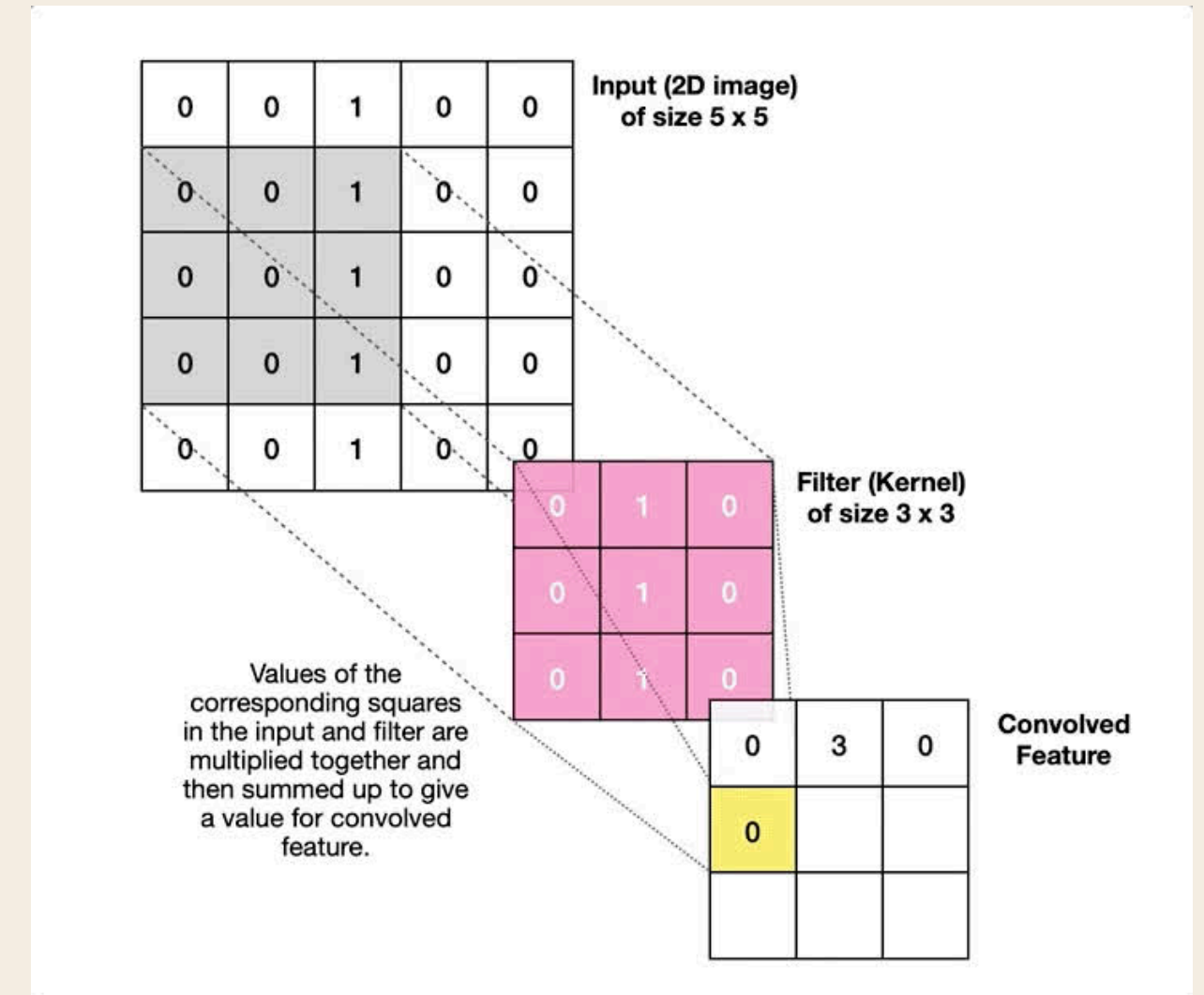
WHAT CNN DOES DIFFERENTLY

1. Local connectivity

- Instead of connecting every neuron to every pixel, each neuron only looks at a small local patch (e.g. 3×3).
- This makes sense because nearby pixels are related

2. Weight sharing

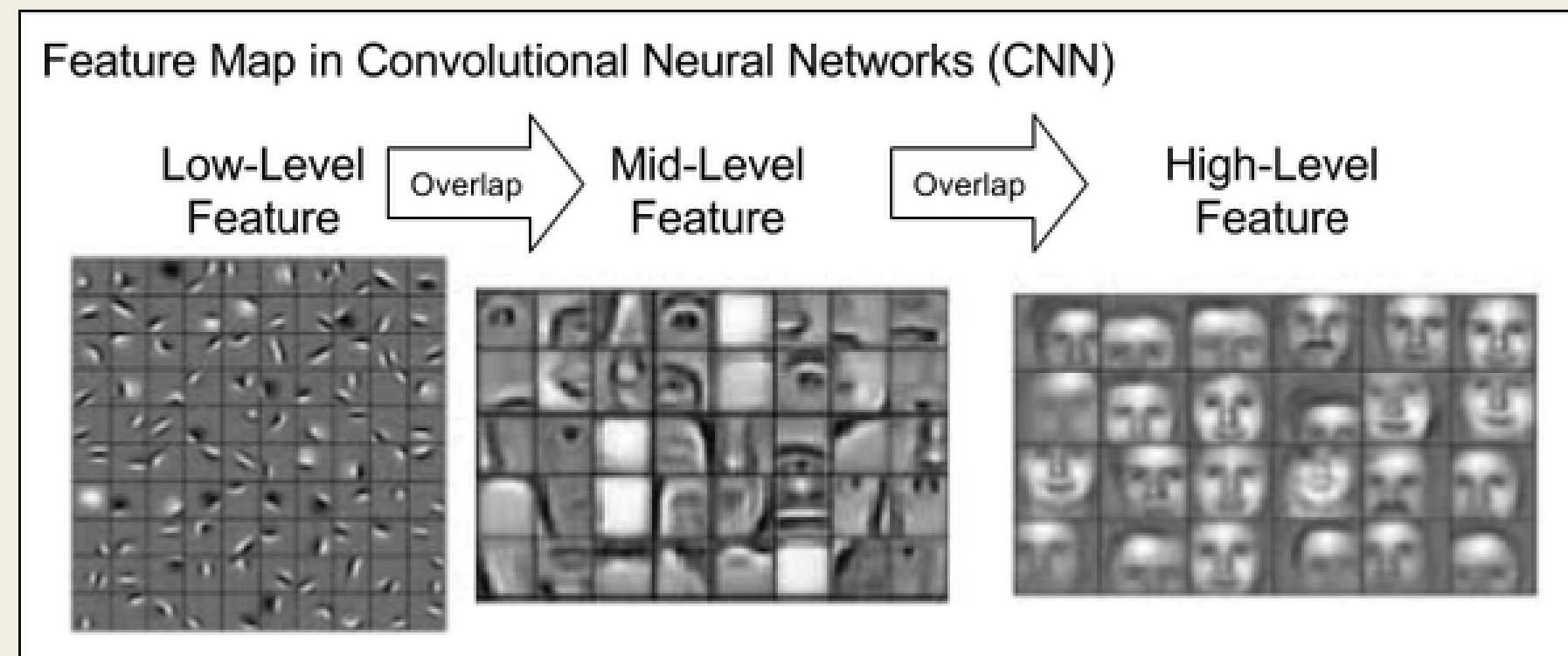
- The same **kernel** (set of weights) is applied across the entire image
- Eg. **A kernel that detects eyes** works the same way whether that eyes is in the top left or bottom right (massively reduces parameters!)



WHAT CNN DOES DIFFERENTLY

3. Spatial hierarchy

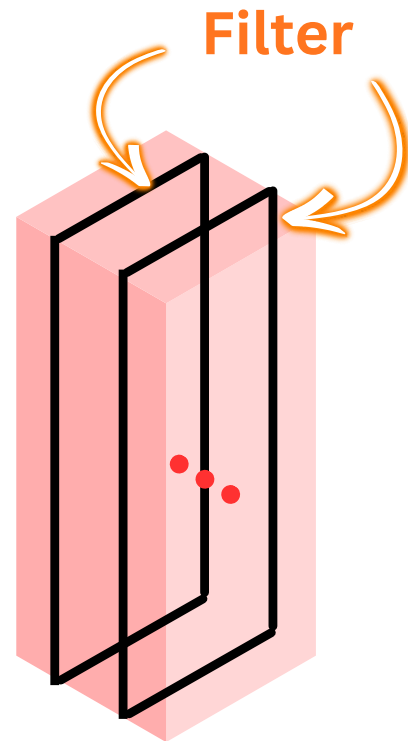
- Early layers apply filters directly to raw pixels, so they can only detect **simple local patterns like edges and corners**
- Later layers apply filters to the output of previous layers, so they're detecting **patterns of patterns**. This is how complexity builds up layer by layer.



Note: When we say low-level, mid-level, high-level features etc. We don't actually know if it's "eyes" or "mouth". So this is just for your intuitive understanding!!

TYPES OF CNN LAYERS

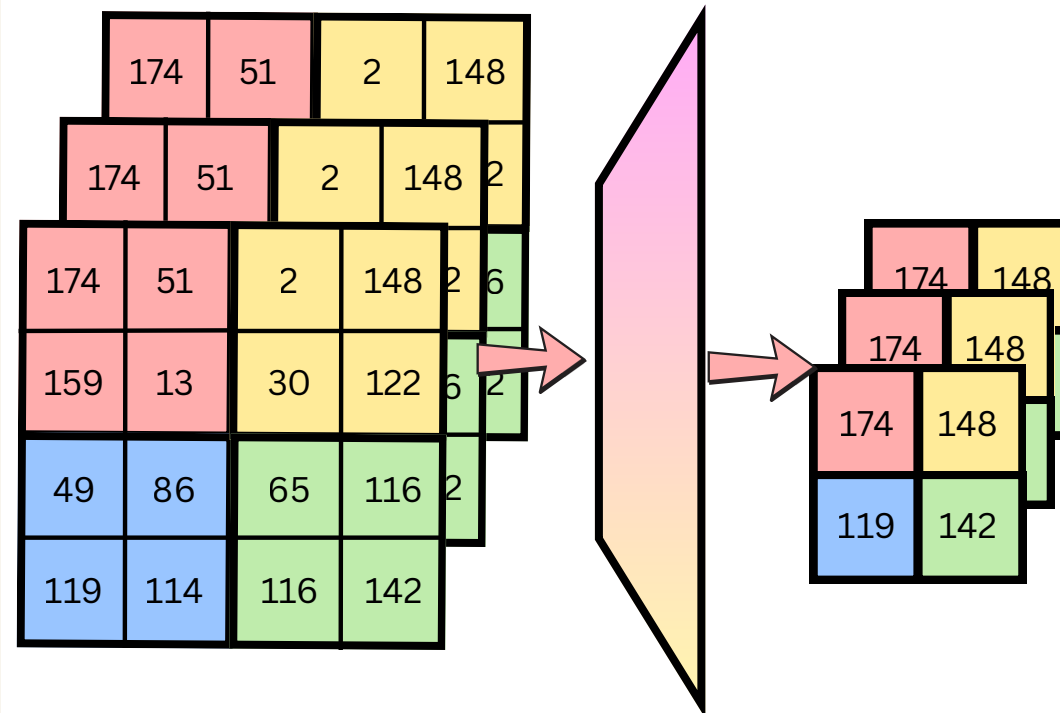
Convolutional Layer



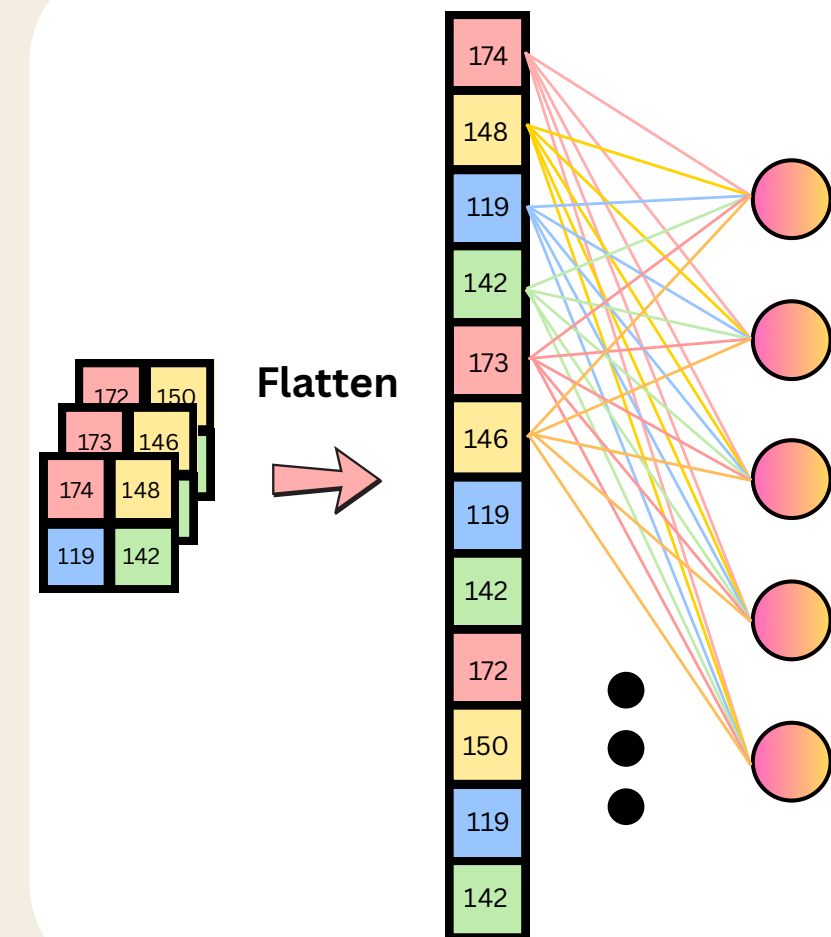
Each convolutional layer can have **different number of filters** which learn different feature.

Pooling Layer

2 x 2 Max Pooling
Stride 2 x 2



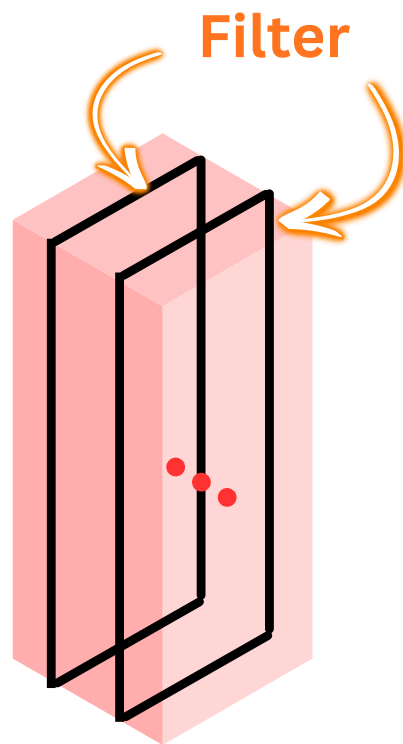
Fully Connected Layer



Let's quickly go through them one by one!

CONVOLUTIONAL LAYER

Convolutional Layer

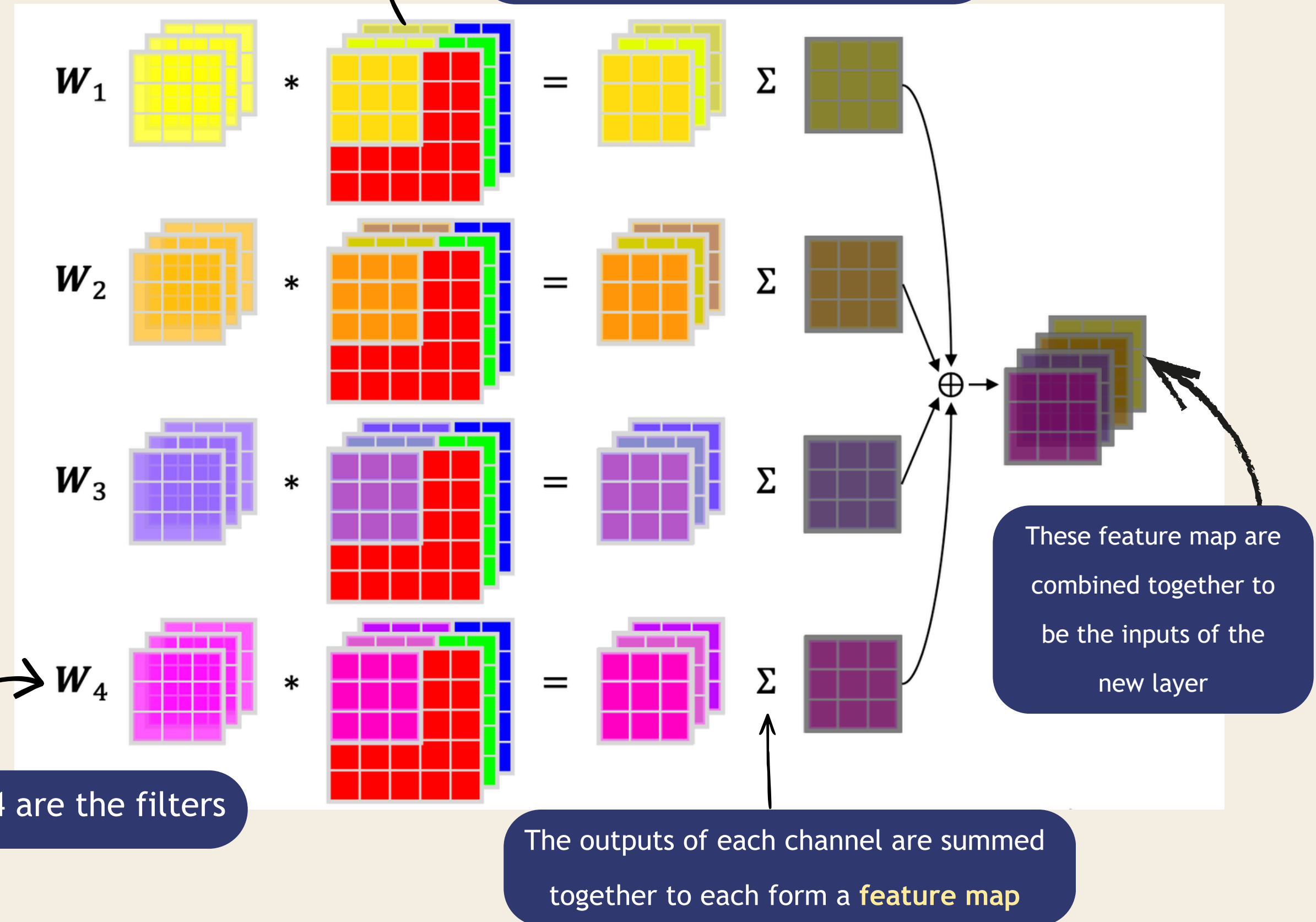


Each convolutional layer can have **different number of filters** which learn different feature.

ps: each “layer” of the filters are called “kernel”

Inside one layer:

these filters (W) stride through the image (broken down by RGB)



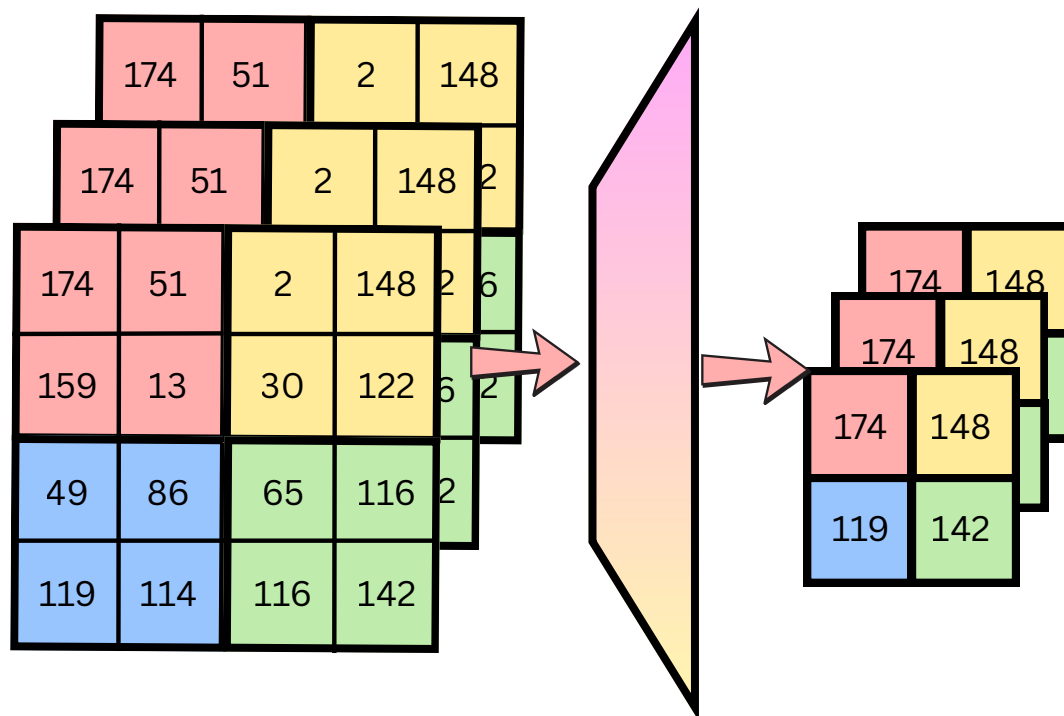
POOLING LAYER

Why do we pool?

- **Reduce spatial size:** shrinks feature maps size, which cuts computation in later layers
- **Reduce overfitting:** fewer spatial dimensions = fewer parameters in later layers

Pooling Layer

2 x 2 Max Pooling
Stride 2 x 2



Pool size → → Stride

-4	0	-2	4	1
3	1	0	2	1
1	0	1	1	1
4	6	5	1	0
-1	2	0	0	0

Max Pooling

3	4
6	5

Min Pooling

-4	-2
-1	0

Average Pooling

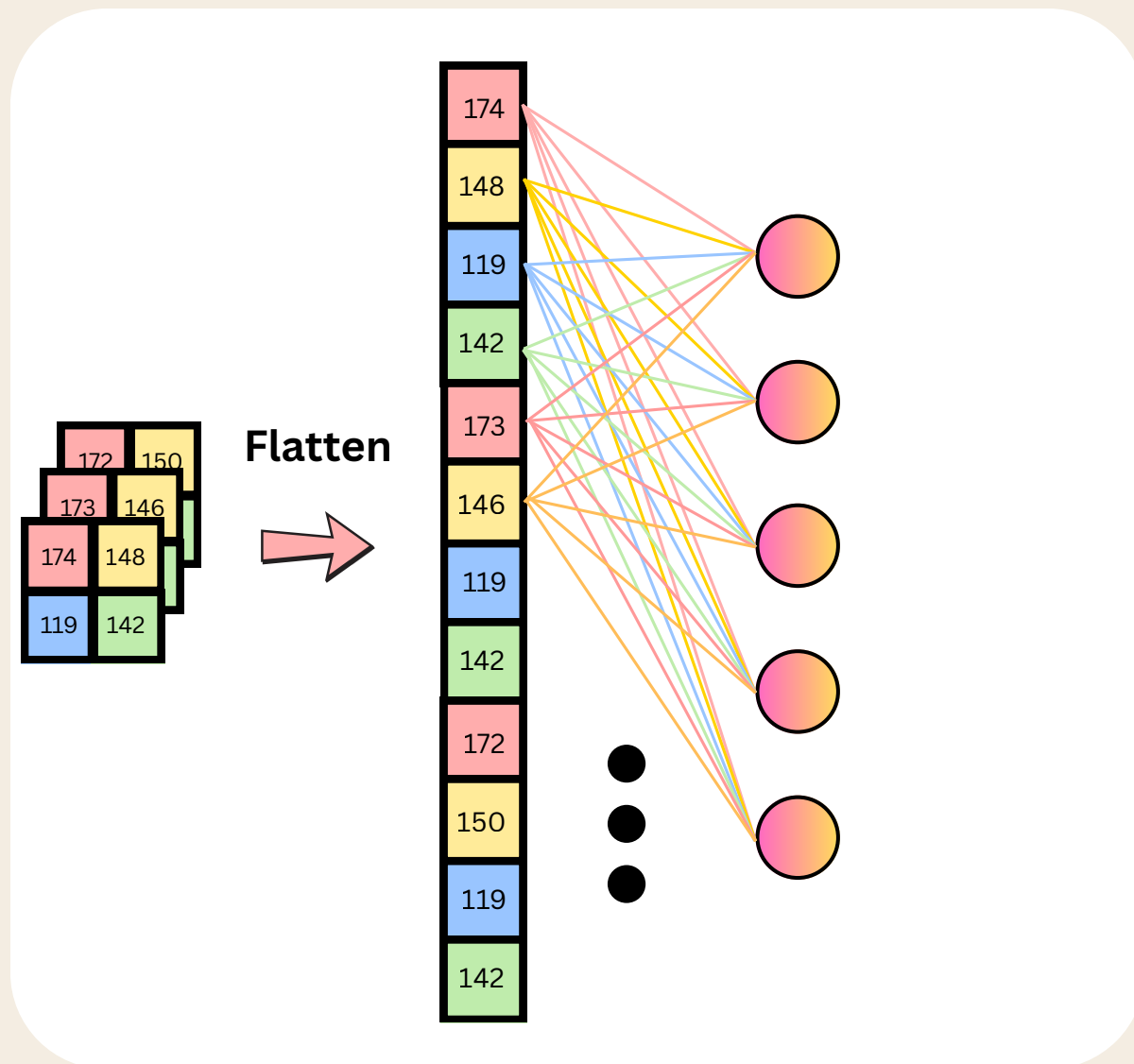
0	1
2	1

Features

Output

FULLY CONNECTED (FC) LAYER

Fully Connected Layer



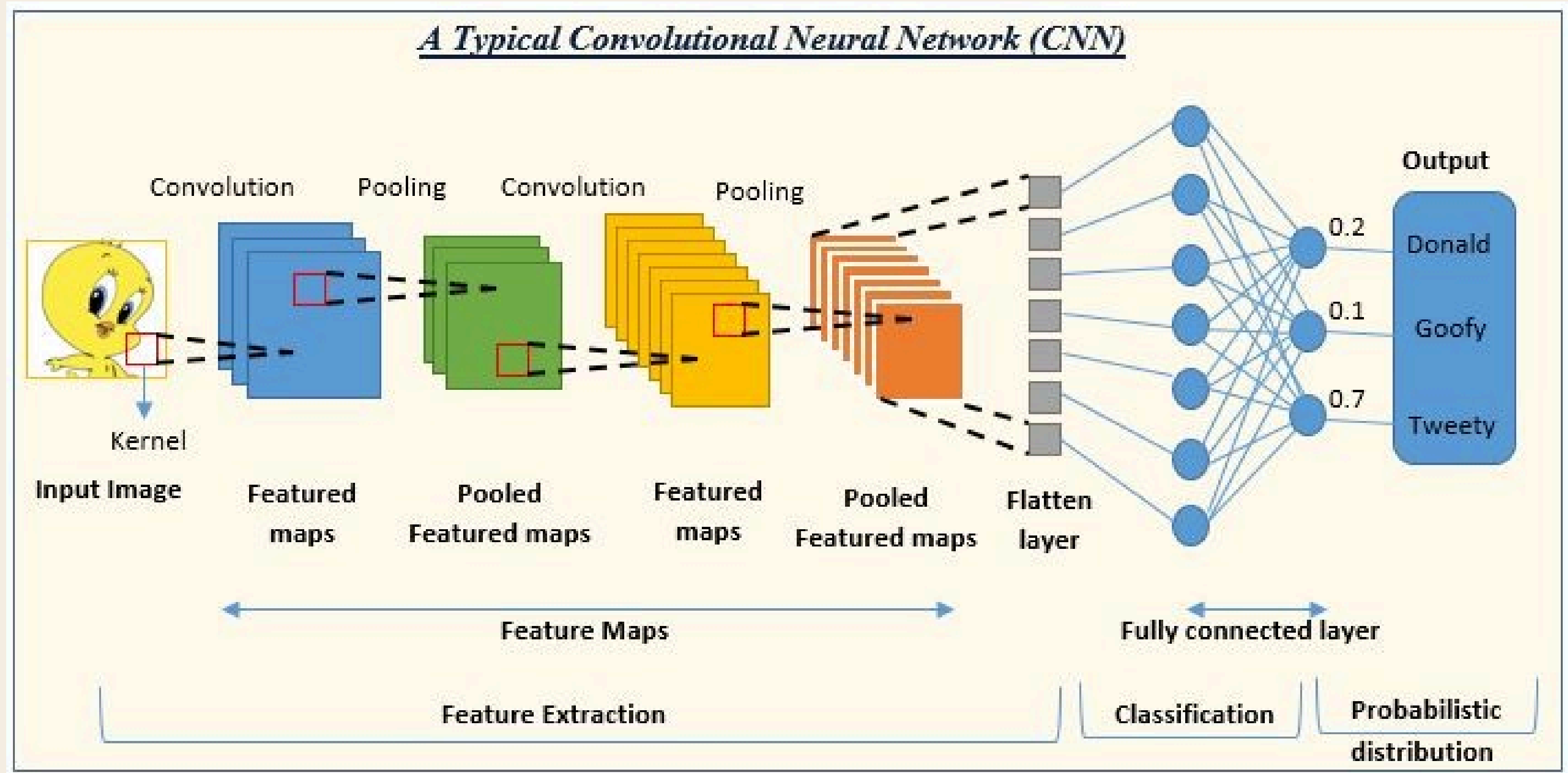
What it does

- Takes the final feature maps and **flattens them into a 1D vector**
- Passes through a **regular ANN**, every neuron connects to every other neuron
- Final layer outputs class probabilities

Key: Conv + Pool layers → feature extraction ("what features exist and where")

Fully Connected layer → classification ("what is this image")

CNN OVERALL STRUCTURE



DEEP LEARNING FOR NLP

Properties of Natural Language:

1. Comprises sequences of words

"The dog bites the man" and "The man bites the dog"

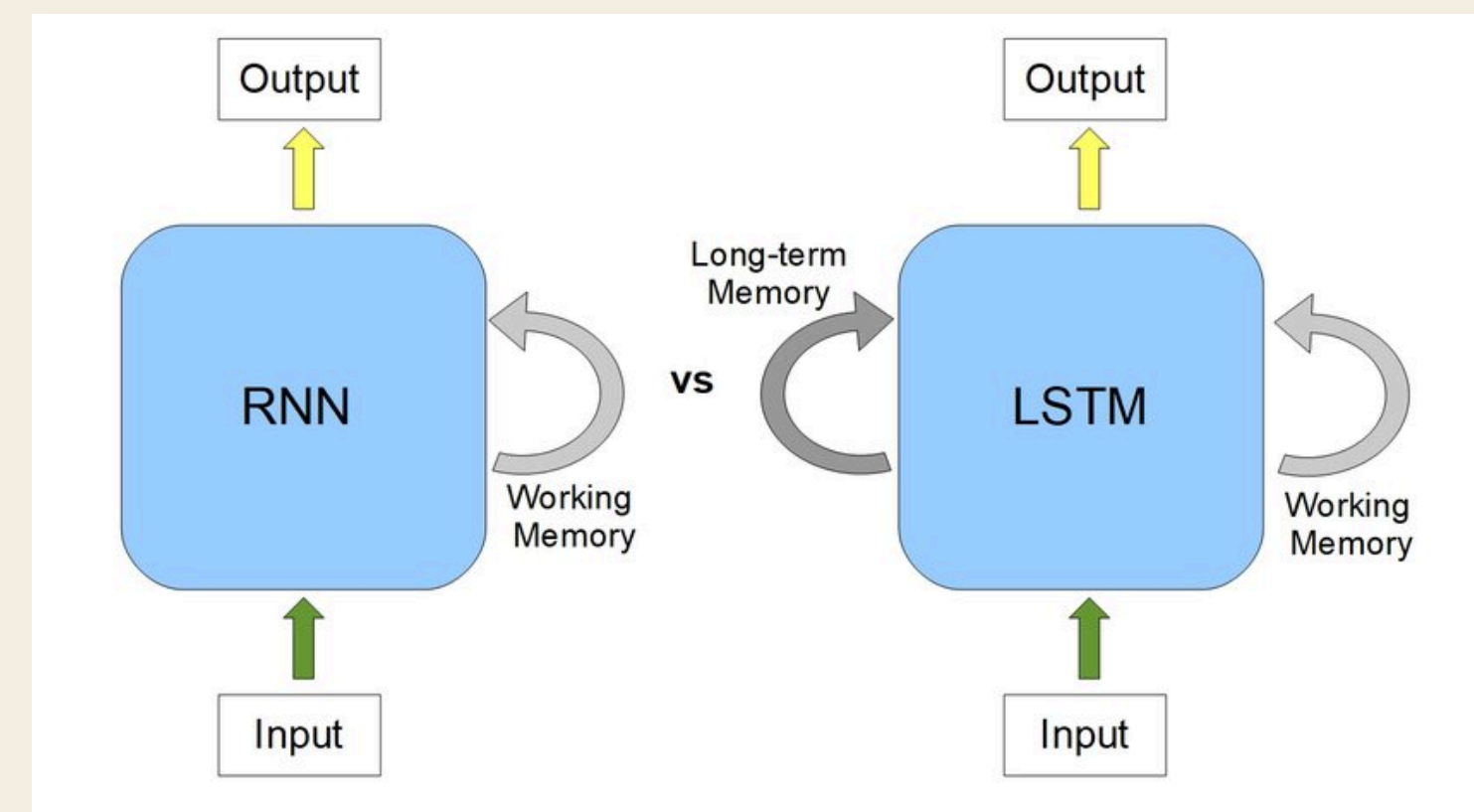
2. Grammatically/semantically depends on the previous words used

Framework architecture:

- Recurrent Neural Networks (RNN)
- Long Short-term Memory (LSTM)

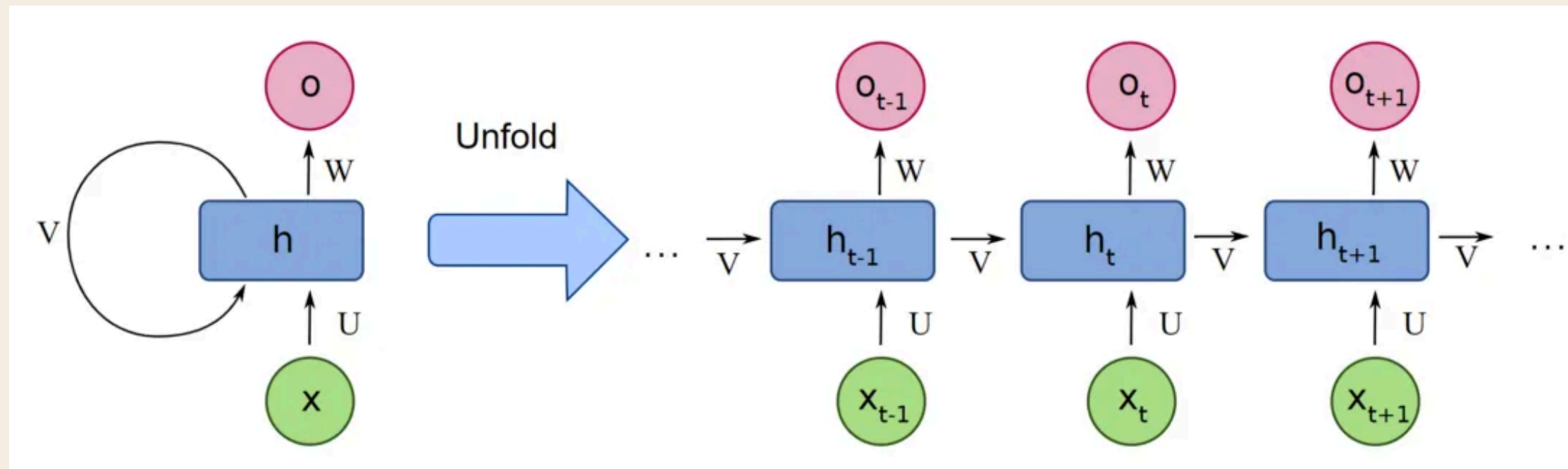
Why other architecture over ANN?

ANN treats each word independently with no sense of **order or context** (same problem as using ANN for images)



RECURRENT NEURAL NETWORK (RNN)

A neural network designed to process sequences by maintaining a hidden state (aka a memory/summary of what it has seen so far)



Note: Same weights (U , V , W) are reused at every time step

How it works

- Processes one **token** at a time, left to right
- At each step it takes two inputs: the current token + the hidden state from the previous step
- Produces an output and updates the hidden state
- The hidden state carries information forward through the sequence

Many words map to one token, but some don't: indivisib
le. Unicode characters like emojis may be split into m
any tokens containing the underlying bytes: 🤝
Sequences of characters commonly found next to each ot
her may be grouped together: 1234567890

RECURRENT NEURAL NETWORK (RNN)

Limitations

- Fails when prediction has long-term dependencies - previous info might be “forgotten” (like humans!)

"I grew up in France. I went to school there. I made many friends. I loved the culture. I speak fluent _____"

Answer is “French” but that information came from the first sentence.

A vanilla RNN has largely forgotten "France" because the hidden state has been overwritten so many times.

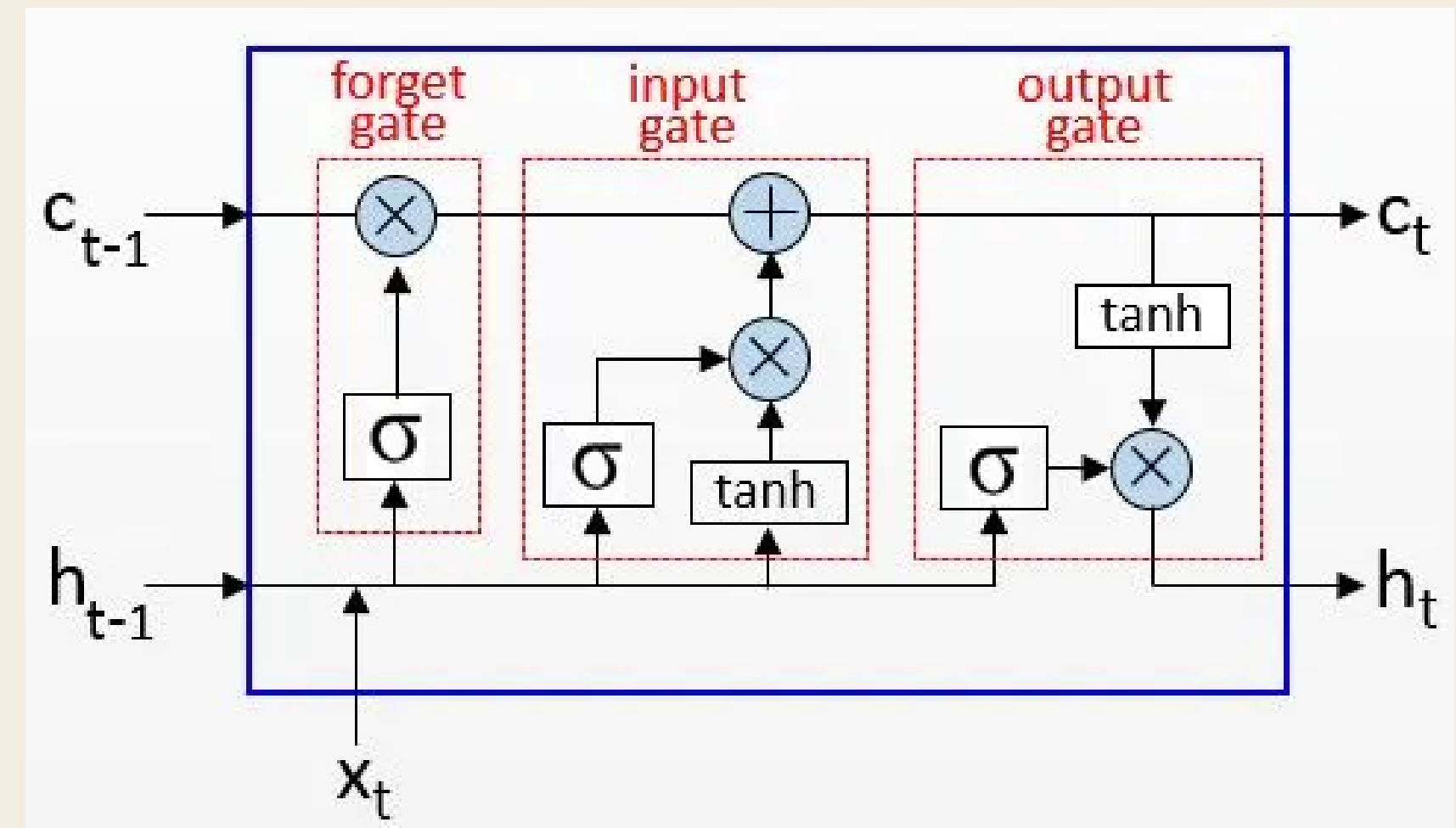
- Sequential processing - tokens must be processed one at a time (slow)
- Vanishing Gradient problem, due to many time steps
 - Each token is a timestep, a long sentence can have very deep RNN

LONG SHORT-TERM MEMORY (LSTM)

A special type of RNN designed to remember information over longer sequences by controlling what to keep, what to forget, and what to output

RNN vs LSTM (Key Diff)

- RNN has **one** hidden state (short-term memory only)
- LSTM has **two**: hidden state (short-term) + cell state (long-term)

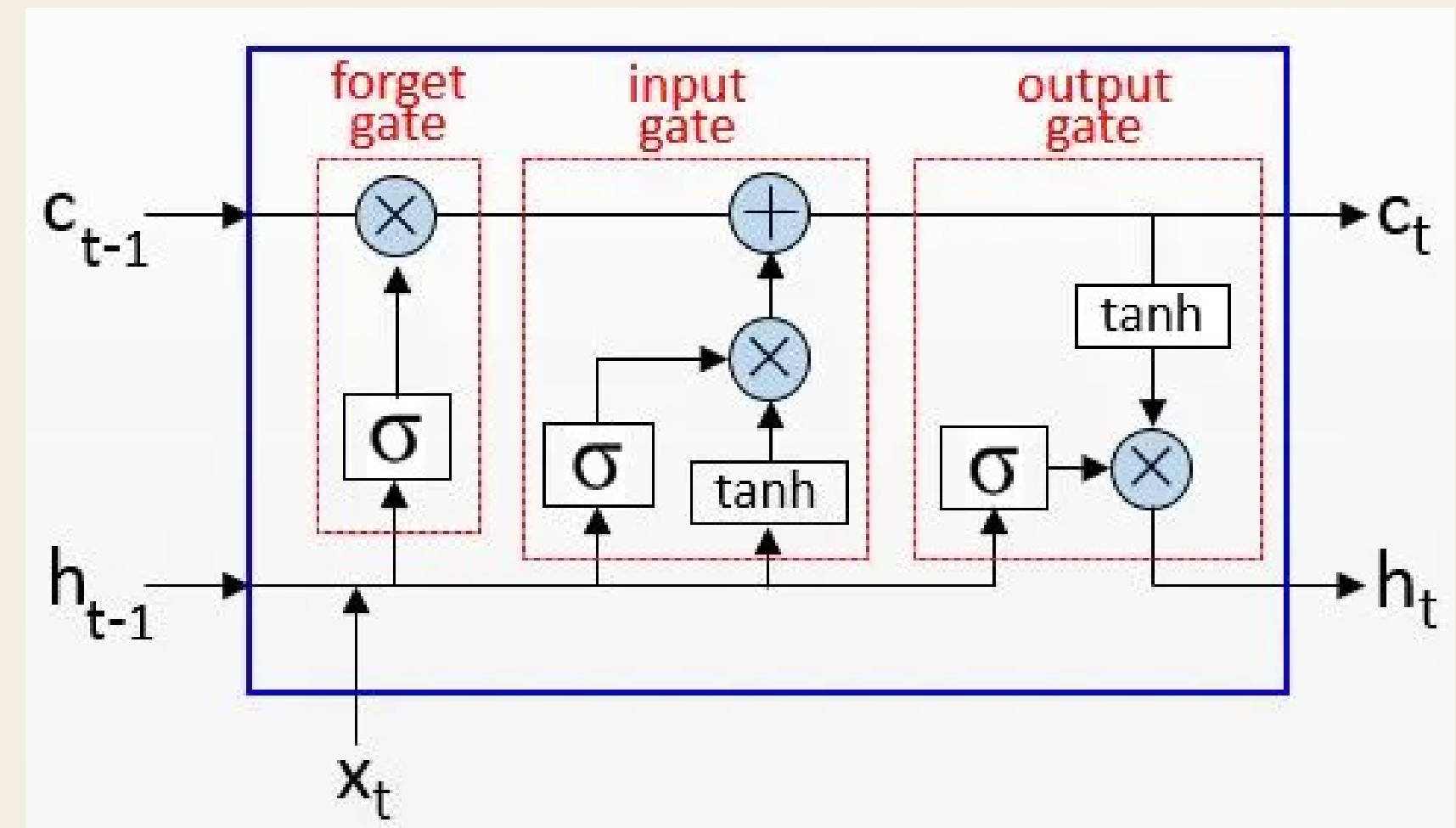


LONG SHORT-TERM MEMORY (LSTM)

A special type of RNN designed to remember information over longer sequences by controlling what to keep, what to forget, and what to output

LSTM consists of 3 gates:

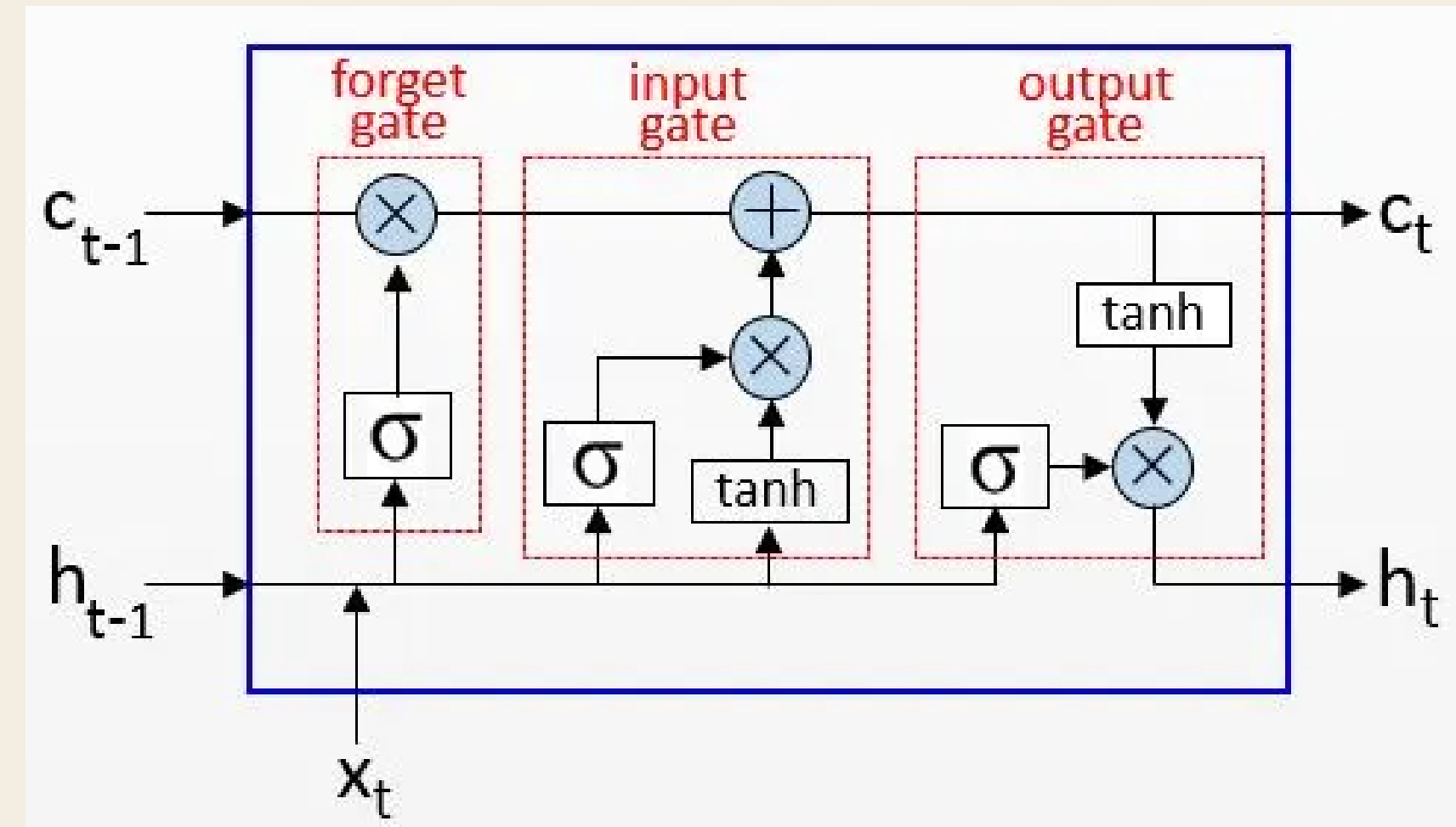
- **Forget gate:** decides what info to discard that is unimportant from the previous time step
- **Input gate:** Decides which information from the input to be used to update the memory state
- **Output gate:** Decides what to output based on input and the memory of the cell



LONG SHORT-TERM MEMORY (LSTM)

Forget gate

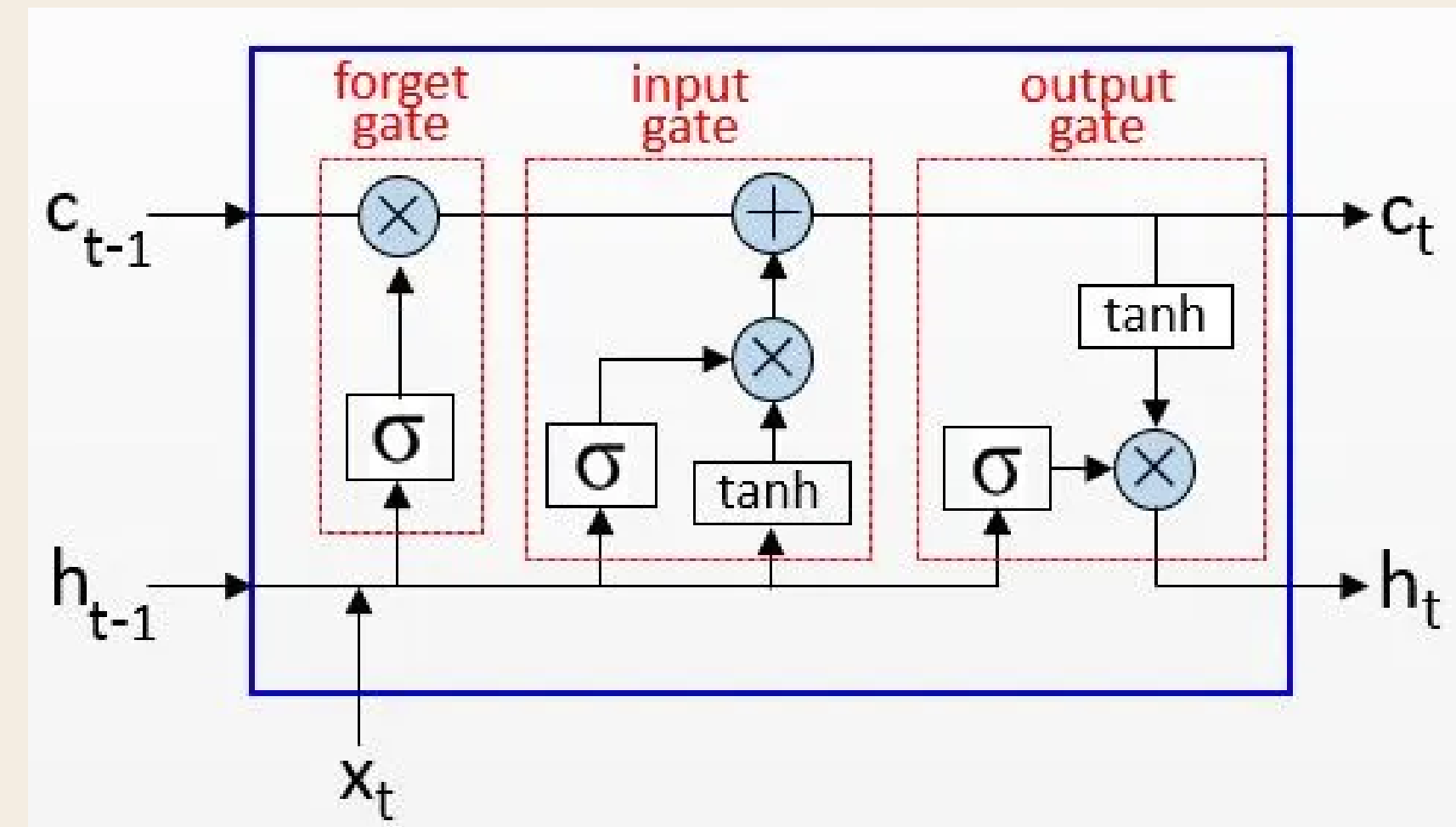
- σ (sigmoid) takes h_{t-1} and $x_t \rightarrow$ outputs values between 0 and 1
- Multiplied ($\times$) with c_{t-1} (previous cell state)
- 0 = completely forget, 1 = completely keep
- This decides how much of the old long-term memory to preserve



LONG SHORT-TERM MEMORY (LSTM)

Input gate

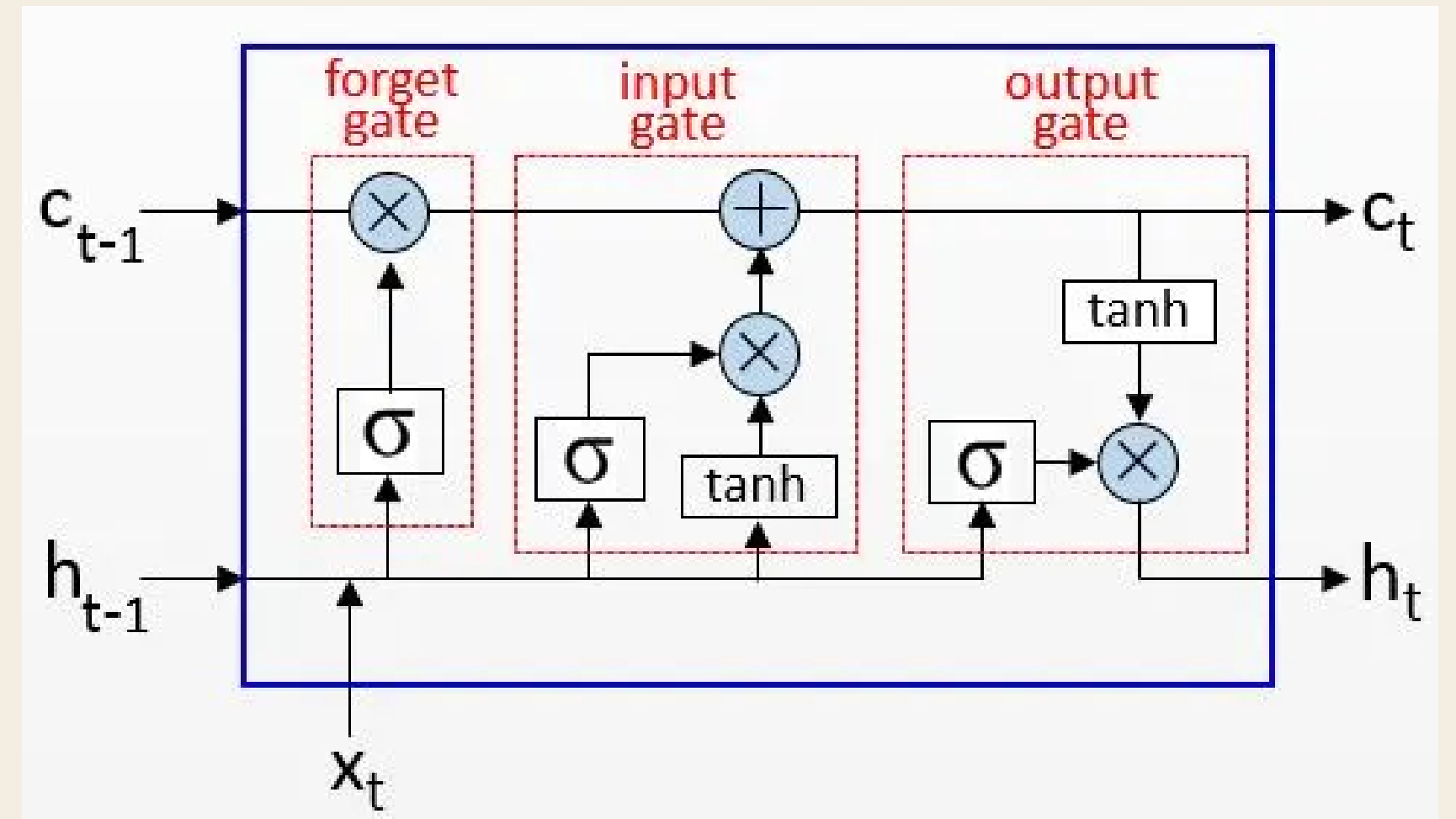
- Two operations happening in parallel:
- σ takes h_{t-1} and $x_t \rightarrow$ decides how much new info to write (0 to 1)
- $\tanh$ takes h_{t-1} and $x_t \rightarrow$ creates candidate values to write (-1 to 1)
- They get multiplied ($\times$) together $\rightarrow$ only the important new info passes through
- Added (+) to the surviving cell state from forget gate $\rightarrow$ produces new c_t



LONG SHORT-TERM MEMORY (LSTM)

Output gate

- σ takes h_{t-1} and $x_t \rightarrow$ decides what to expose from cell state (0 to 1)
- $\tanh$ squishes c_t between -1 and 1
- They get multiplied ($\times$) $\rightarrow$ produces new h_t



TUTORIAL QUESTIONS

TUTORIAL QUESTION 1A, 1B

What is a pretrained model (in Deep Learning)? And why is it important?

Anyone wants to share?

Transfer learning be like



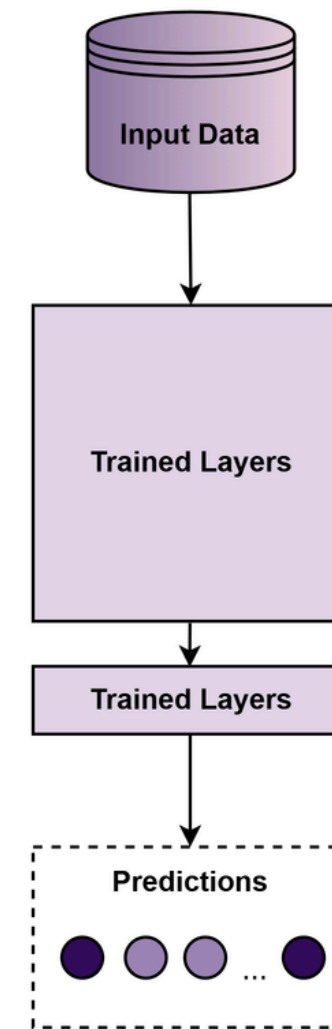
TUTORIAL QUESTION 1A, 1B

Pretrained model

- a model that has been trained on a large dataset for general task
- and its learnt weights (or part of it) are reused for a new task

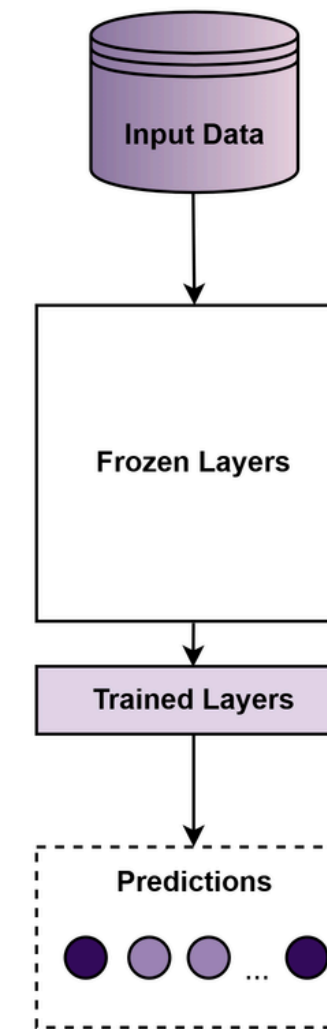
Training from Scratch

Train All Layers of the Model

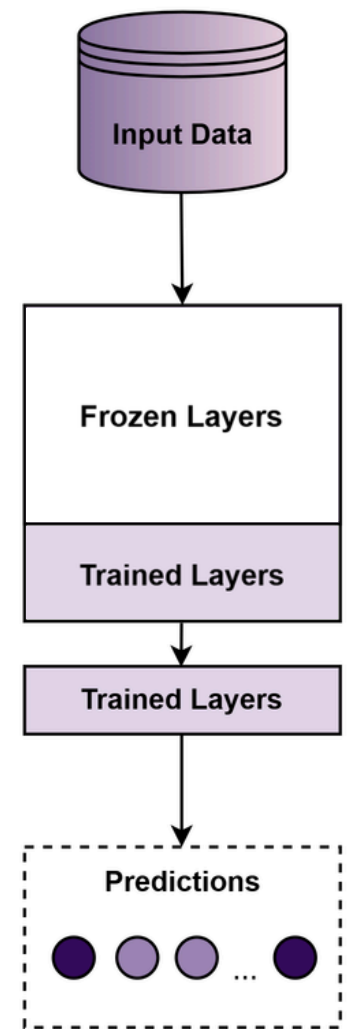


Training a Pre-Trained Model

Use the Model Out-of-the-Box



Fine-Tune the Model



TUTORIAL QUESTION 1A, 1B

Why are Pre-trained models important?

Instead of training a model from scratch (**which requires massive data, compute, and time**). You take a model that already "knows" a lot about the world (patterns, features, language structure, visual concepts) and adapt it to your specific problem.

So essentially, pre-trained models:

- saves significant time and computational resources,
- enables high-accuracy results even with limited labeled data



TUTORIAL QUESTION 1C

Give one example of how you would use a pretrained model for an image classification task.

Model	Year	Number of Parameters	Top-1 Accuracy
VGG-16	2014	138 Million	74.5%
ResNet-50	2015	25 Million	77.15%
Inception V3	2015	24 Million	78.8%
EfficientNetB0	2019	5.3 Million	76.3%
EfficientNetB7	2019	66 Million	84.4%

Anyone wants to share?

TUTORIAL QUESTION 1C



My example

Say you want to build an app that tells you what breed of dog is in a photo.

Choose VGG16/ResNet50, a model pretrained on ImageNet (1.2 million). But it was trained to classify 1,000 generic ImageNet categories, not dog breeds specifically.

1. Swap the final classifier layer. Replace the 1,000 class output with a new layer that has 120 outputs (one per dog breed)
2. Freeze all the pretrained layers so you don't destroy what the pre-trained already knows (weights)
3. Train just the new classifier on your dog breed dataset
4. (Optional) Fine-tune: unfreeze the later layers and retrain with a very small learning rate to squeeze out more accuracy

TUTORIAL QUESTION 1D

Why is training a deep learning model from scratch often impractical?

1. Requires large amount of labeled data

- A deep model has a lot of parameters to learn (*think about us counting the no of weights for pollev question last week*)
- To learn them reliably you need A LOT of labeled examples

2. Requires enormous compute and a lot of time

- Training ResNet from scratch on ImageNet takes days on multiple high-end GPUs.
- Most individuals and small teams simply don't have access to that kind of hardware or budget.
- Models like GPT took millions of dollars of compute to train (today billions)

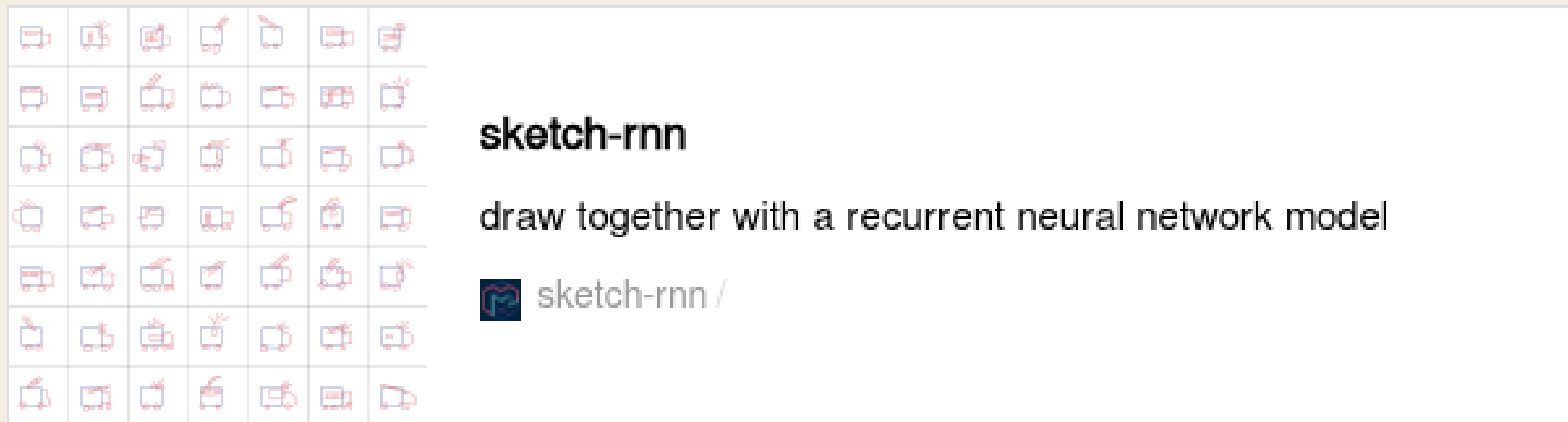
TUTORIAL QUESTION 2

(a) What is a generative model?

(b) Explain the roles of a Generator and Discriminator in GAN

Allow me to show you this fun website!

https://magenta.tensorflow.org/assets/sketch_rnn_demo/index.html

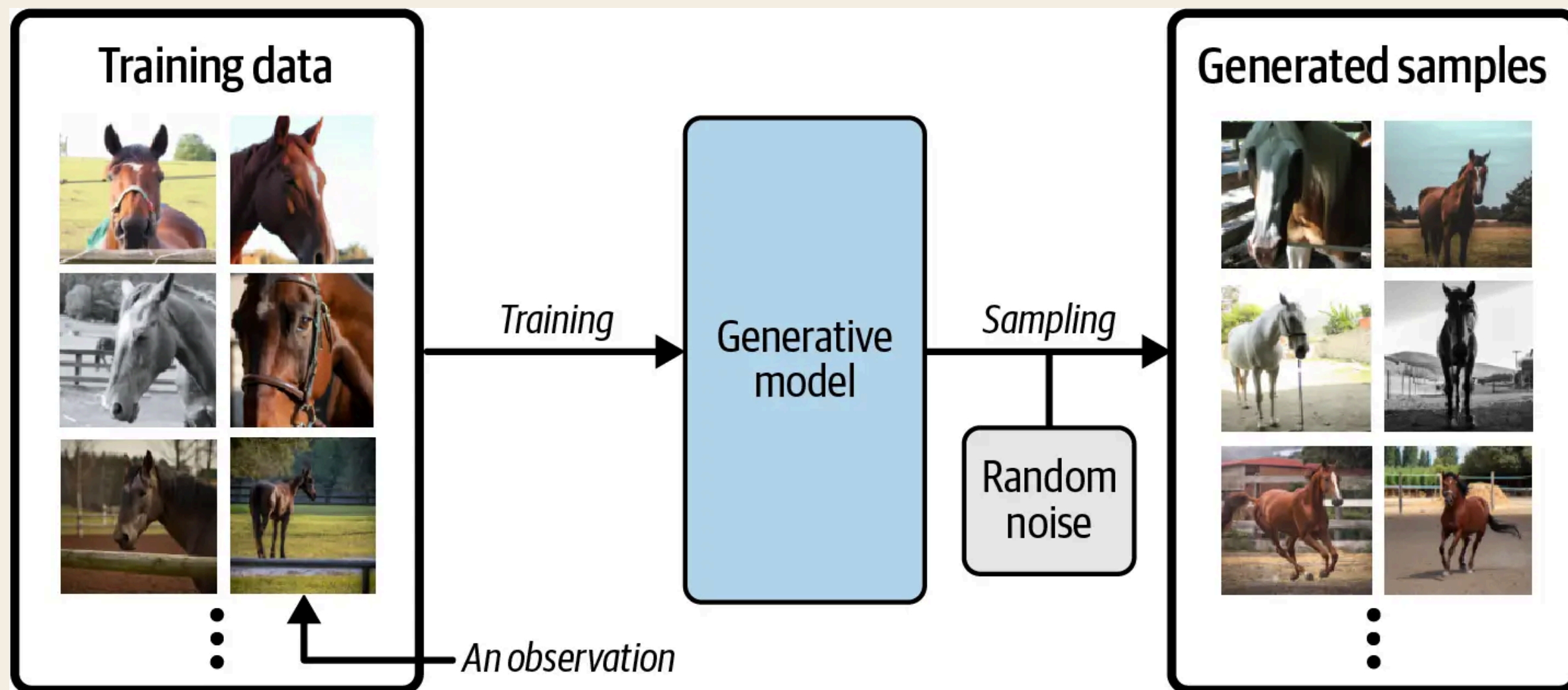


this uses VAE (another generative model) + RNN

TUTORIAL QUESTION 2

Generative Model

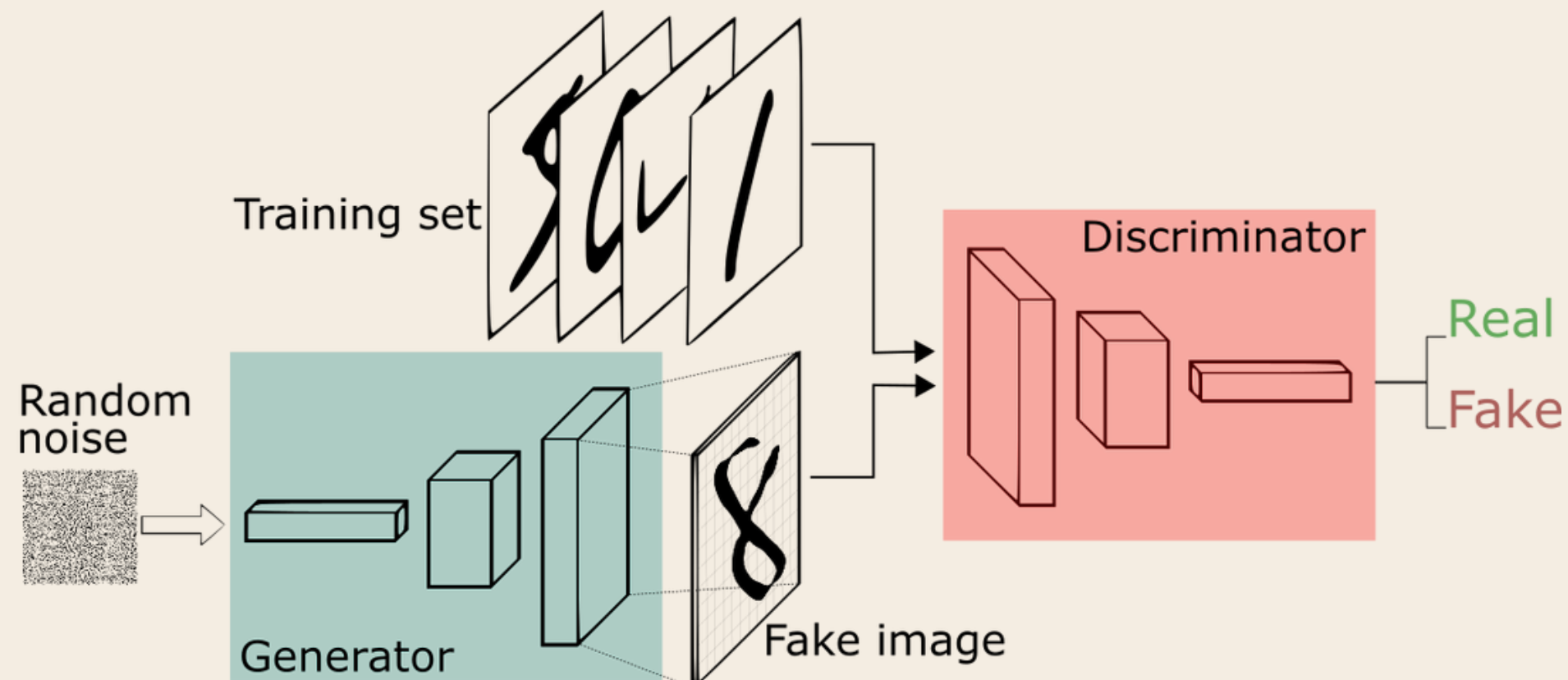
A model that learns to **create new data** that resembles its training data (by learning the underlying data distribution)



TUTORIAL QUESTION 2

Generator and Discriminator in GAN (Generative Adversarial Network).

- Unsupervised ML technique
- **Generator:** Takes random noise as input and produces fake data
 - Goal: fool the Discriminator into thinking its output is real
- **Discriminator:** Tries to distinguish between real and fake data
 - Goal: correctly distinguish real data from the Generator's fakes



As we repeat, network improves because of each other

Training ideally stops when the Discriminator outputs 50/50 probability (can't differentiate whether real or fake)

TUTORIAL QUESTION 3

What are common ways to reduce overfitting in deep neural networks?

- Train with more data
 - More data → too much to memorize, forced to find patterns that actually generalize
- Data augmentation techniques
 - expanding your training dataset by applying transformations to existing data
 - so the model sees more variety without needing to collect more data
 - eg. flipped, rotated, cropped versions of the same image
- Add noise to input data
 - if the input keeps changing slightly, the model can't memorize exact inputs
- Regularization (L1/L2)
 - Adds a penalty to the loss function for having large weights
 - Regularization forces weights to stay small (or 0), keeping the model simpler and general

TUTORIAL QUESTION 3

What are common ways to reduce overfitting in deep neural networks?

- **Add Dropout layers**
 - Randomly turns off neurons during training.
 - This stops neurons from **co-adapting** (one neuron relies too heavily on another)
Each neuron is forced to learn independently useful features
- **Early stopping during the training**
 - Watches the validation loss
 - When it stops improving and starts getting worse, you stop training
- **Cross-validation**
 - Ensures the model is evaluated on every part of the data, not just one fixed split
 - Prevents you from accidentally tuning your model to one specific test set

TUTORIAL QUESTION 4A

What is Generative AI and how is it different from traditional machine learning?

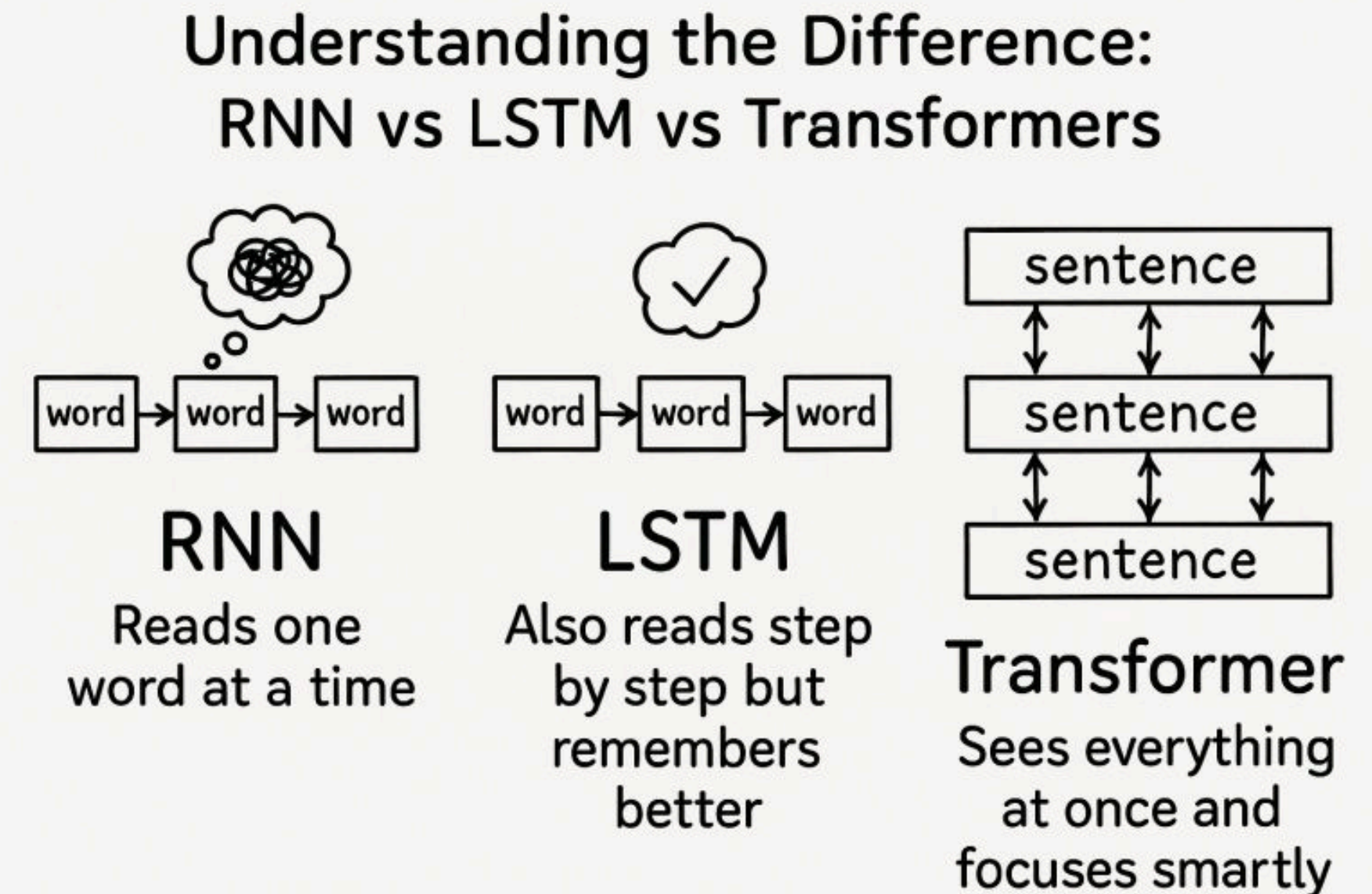
Characteristic	AI	Traditional ML	Generative AI
Purpose	Develop computer systems that can perform tasks that typically require human intelligence.	Make predictions or decisions based on given data.	Generate new data samples that resemble a given set of training data.
Data Interaction	Models use various techniques and strategies designed to mimic human intelligence across a wide range of applications.	Models learn from data to make predictions or decisions on new unseen data.	Models produce new data that weren't part of the original dataset but share similar characteristics.

TUTORIAL QUESTION 4B

What are the limitations of RNN/LSTM models, and how do Transformer models address these limitations?

Limitations of RNN/LSTM

- Difficulty in capturing long-range dependencies in sequences (forgetful)
- Sequential processing → slow training
- Prone to vanishing gradient problems even though LSTM mitigates this partially



TUTORIAL QUESTION 4B

What are the limitations of RNN/LSTM models, and how do Transformer models address these limitations?

Difficulty in capturing long-range dependencies in sequences (forgetful).

- Transformers use **self-attention** (Every token attends directly to every other token regardless of distance - Token 500 can directly reference token 1)

Sequential processing → slow training

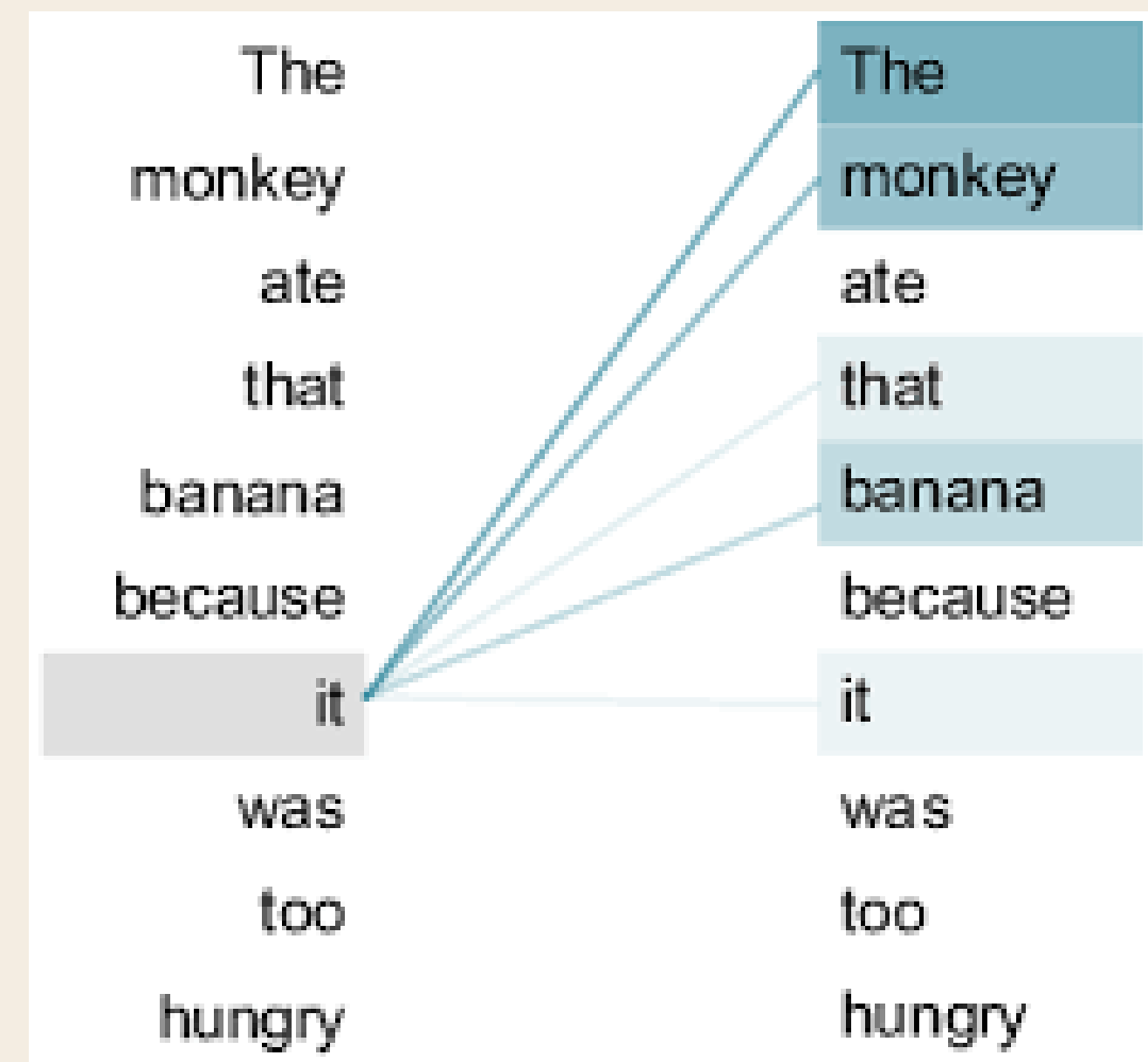
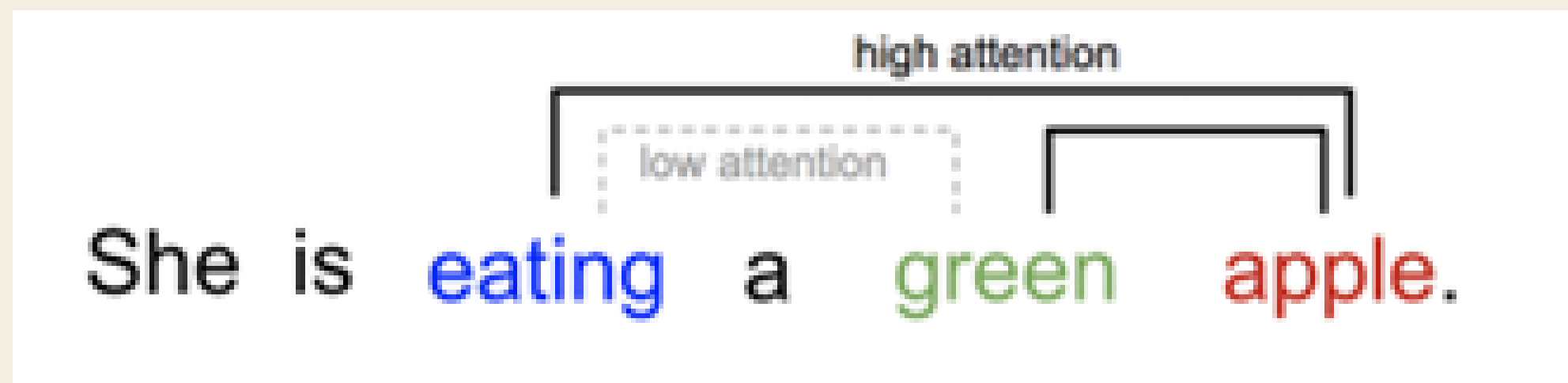
- Transformers process all tokens simultaneously in parallel (No need to wait for previous tokens)

Prone to vanishing gradient problems

- Self-attention creates direct connections between any two tokens
- Gradients don't have to travel through hundreds of time steps to get back to early token (Gradient stays healthy :D)

TUTORIAL QUESTION 4B

Self-attention **assigns different weights** to all other words in the sequence relative to each word being processed, capturing which words are most relevant to understanding it.



**SHARING BY
AARON AND
CHENG HOU**

EXTRA MATERIALS

1. Regularization (L1/L2)

<https://medium.com/@pierre.lgsm/regularization-techniques-in-deep-learning-bdf06862a5ff>

2. CNN (by StatQuest!)

<https://www.youtube.com/watch?v=HGwBXDKFk9I&t=1s>

3. RNN (by StatQuest!)

<https://www.youtube.com/watch?v=AsNTP8Kwu80&t=307s>

4. LSTM (by StatQuest!)

<https://www.youtube.com/watch?v=YCzL96nL7j0>

5. Transformers (by StatQuest!)

<https://www.youtube.com/watch?v=zxQyTK8quyY>